

8강

홈화면 만들기





spring

- com.green.member-join 프로젝트를 생성한다.

The screenshot shows the 'New Spring Starter Project' dialog. The 'Artifact' field is highlighted with a red rectangle, and a red arrow points from it to the 'Artifact 란?' box. The dialog contains the following fields:

- Service URL: `https://start.spring.io`
- Name: `com.green`
- ☒ Use default location
- Location: `D:\Spring_Boot\pjt\com.green` (with a 'Browse' button)
- Type: `Gradle - Groovy` (dropdown)
- Packaging: `Jar` (dropdown)
- Java Version: `21` (dropdown)
- Language: `Java` (dropdown)
- Group: `com.green`
- Artifact: `member-join` (highlighted)
- Version: `0.0.1-SNAPSHOT`
- Description: `Demo project for Spring Boot`
- Package: `com.green`
- Working sets: ☐ Add project to working sets (with 'New...' and 'Select...' buttons)

Artifact 란?

- Artifact = 이 프로젝트의 "프로그램 이름"
- 빌드되면 만들어지는 파일 이름
- 프로젝트 폴더 이름
- 나중에 실행되는 jar 이름
- 전부 여기서 결정된다.



spring

- application.properties 파일의 아래 내용을 작성하여 환경을 설정한다.

```
spring.application.name=com.green  
# Tomcat 서버를 내장되어 있다.  
# 원래 포트번호는 8080인데 => 8090으로 변경함  
# Tomcat  
server.port=8090
```

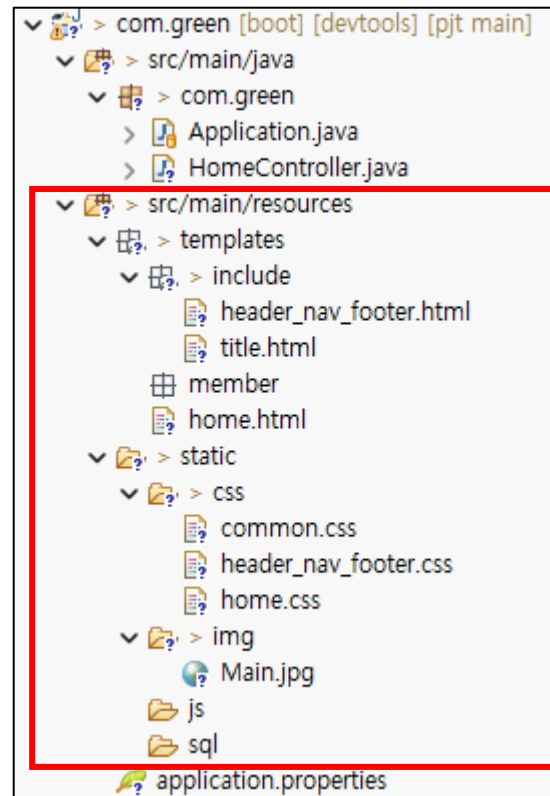
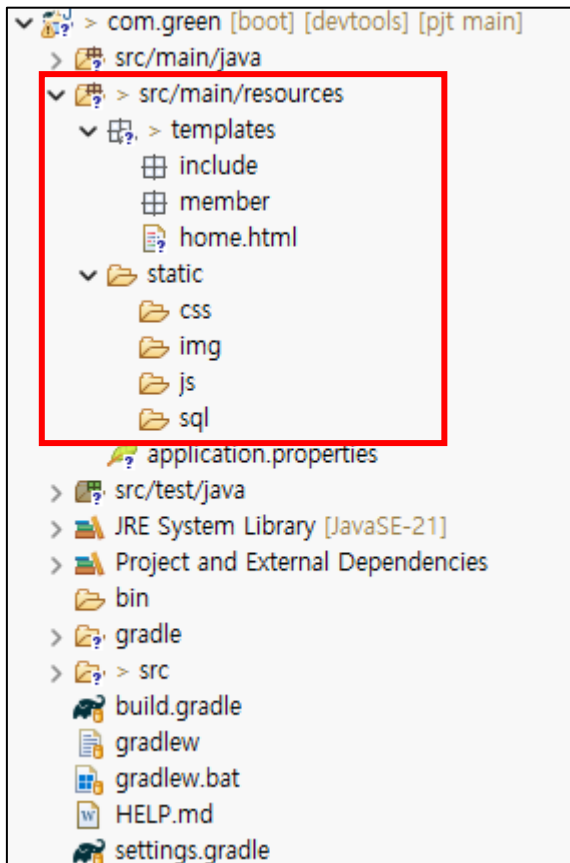
```
# Dev Tools  
spring.devtools.restart.enabled=true
```

```
# Thymeleaf #주석  
spring.thymeleaf.cache=false  
spring.thymeleaf.prefix=classpath:/templates/  
spring.thymeleaf.suffix=.html  
spring.thymeleaf.check-template-location=true
```



spring

- src -> main -> resources -> static 디렉토리에서 정적 파일을 관리 하라
- templates 폴더 아래, include, member 폴더를 생성 한다.
 - static 폴더 아래 css, img, js, sql 폴더를 생성한다.





spring

- src -> main -> resources -> static 디렉토리에서 정적 파일을 관리 하라
 - templates -> home.html 아래 코드대로 기본 홈페이지를 작성한다.

```
<!DOCTYPE html>
<html lang="ko" xmlns:th="http://www.thymleaf.com">
<head>
<meta charset="UTF-8">
<title>Member Join</title>
</head>
<body>
    <header>
        <div id="top">
            <h3>MEMBER JOIN</h3>
        </div>
        <div id="header_wrap">
            <a th:href="@{/}">HOME</a>
            <a th:href="@{/member/signup}">회원가입</a>
            <a th:href="@{/member/signin}">로그인</a>
        </div>
    </header>
    <nav>
        <div id="nav_wrap">
            <a th:href="@{/example}">EXAMPLE</a>
        </div>
    </nav>
```

```
<section>
    <div id="section_wrap">
        <div class="word">HOME</div>
        <div class="content">
            
        </div>
    </div>
</section>
<footer>
    <div id="footer_wrap">Copyright OFFICE @Green</div>
</footer>
</body>
</html>
```



spring

- src -> main -> resources -> static 디렉토리에서 정적 파일을 관리 하라
- static -> css -> common.css, home.css, header_nav_footer.css 파일을 생성한다.
 - 각각의 css 파일을 아래와 같이 작성한다.

home.css

```
@charset "UTF-8";
section {
width: 100%;
padding: 30px 0;
}

#section_wrap {
width: 1000px;
height: 800px;
margin: 0 auto;
}

#section_wrap .word {
text-align: center;
font-weight: bolder;
margin: 20px 0;
}

#section_wrap .content img {
max-width: 1000px;
}h
```

common.css

```
@charset "UTF-8";
* {
margin: 0; padding: 0;
}

a {
color: #404040;
text-decoration: none;
}

li {
list-style: none;
}

input {
width: 300px;
padding: 10px;
border-radius: 5px;
border: 1px solid #ccc;
outline: none;
margin: 3px;
}

input[type='button'],
input[type='reset'] {
width: 150px;
cursor: pointer;
}
```



spring

- src -> main -> resources -> static 디렉토리에서 정적 파일을 관리 하라
 - static -> css -> common.css, home.css, header_nav_footer.css 파일을 생성한다.
 - 각각의 css 파일을 아래와 같이 작성한다.

header_nav_footer.css

```
@charset "UTF-8";
header {
width: 100%;
padding: 7px 0;
}

#top {
height: 70px;
line-height: 70px;
border-bottom: 1px solid #ccc;
text-align: center;
background-color: #ededed;
}

#header_wrap {
width: 1000px; height: 40px;
line-height: 40px;
margin: 0 auto;
text-align: right;
}
```

```
nav {
width: 100%;
border-top: 1px solid #ccc;
border-bottom: 1px solid #ccc;
padding: 10px 0;
}

#nav_wrap {
width: 1000px; height: 50px;
line-height: 50px;
margin: 0 auto;
text-align: right;
}
```

```
footer {
width: 100%;
border-top: 1px solid #ccc;
padding: 10px 0;
background-color: #363636;
}

footer #footer_wrap {
width: 1000px; height: 100px;
line-height: 100px;
color: #fafafa;
margin: 0 auto;
text-align: center;
font-size: .9em;
}
```



spring

- src -> main -> resources -> static 디렉토리에서 정적 파일을 관리 하라
 - 외부의 각각의 CSS 파일을 Link로 연결한다.

```
<!DOCTYPE html>
<html lang="ko" xmlns:th="http://www.thymleaf.com">
<head>
<meta charset="UTF-8">
<title>Member Join</title>
<Link rel="stylesheet" th:href="@{/css/common.css}">
<Link rel="stylesheet" th:href="@{/css/header_nav_footer.css}">
<Link rel="stylesheet" th:href="@{/css/home.css}">
</head>
```



spring

- src -> main -> java -> HomeController.java 클래스를 생성한다.
- Home.html 화면을 return 하는 HomeController 파일을 아래와 같이 작성한다.

```
package com.green;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

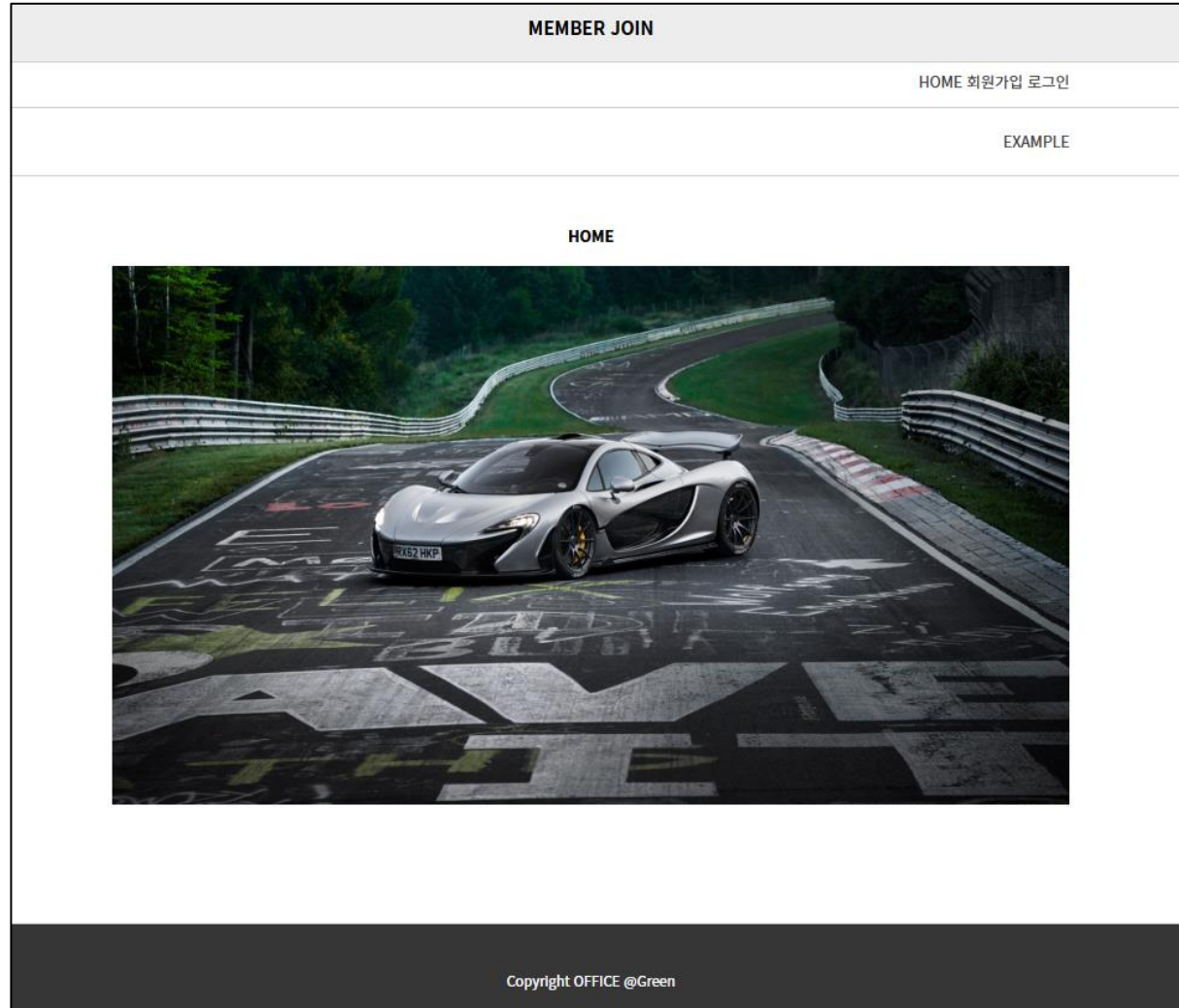
@Controller
public class HomeController {

    @GetMapping({ "", "/" })
    public String home() {
        System.out.println("HomeController");
        return "home";
    }
}
```



spring

➤ 실행화면





spring

- thymleaf의 플래그먼트(=조각)를 이용하여 <header>,<nav>,<footer> 공통요소 분리 및 재활용한다.
- src -> main -> resources -> templates -> include 폴더 생성 후 title.html 파일생성

```
<!DOCTYPE html>
<html lang="ko" xmlns:th="http://www.thymLeaf.com">
<head>
<meta charset="UTF-8">
<title th:fragment>Member Join</title>
</head>
</html>
```

- **th:fragment**속성을 이용하여 title를 조각으로 분리하였다.
- Fragment를 사용하려면 **th:replace** 속성을 이용한다.



spring

- thymleaf의 플래그먼트(=조각)를 이용하여 <header>, <nav>, <footer> 공통요소 분리 및 재활용한다.

- src -> main -> resources -> templates -> include 폴더 생성 후 header_nav_footer.html 파일생성

```
<!DOCTYPE html>
<html lang="ko" xmlns:th="http://www.thymleaf.com">
  <header th:fragment="header">
    <div id="top">
      <h3>MEMBER JOIN</h3>
    </div>
    <div id="header_wrap">
      <a th:href="@{/}">HOME</a>
      <a th:href="@{/member/signup}">회원가입</a>
      <a th:href="@{/member/signin}">로그인</a>
    </div>
  </header>
  <nav th:fragment="nav">
    <div id="nav_wrap">
      <a th:href="@{/example}">EXAMPLE</a>
    </div>
  </nav>
  <footer th:fragment="footer">
    <div id="footer_wrap">Copyright OFFICE @Green</div>
  </footer>
</html>
```



spring

➤ thymleaf의 플래그먼트(=조각)를 이용하여 <header>,<nav>,<footer> 공통요소 분리 및 재활용한다.

- src -> main -> resources -> templates -> home.html 파일을 thymleaf의 플래그먼트로 분리한 조각들을 이용해 다시 작성한다.

```
<!DOCTYPE html>
<html lang="ko" xmlns:th="http://www.thymleaf.com">
<head>
<meta charset="UTF-8">
<title th:replace="~/include/title.html::title">Member Join</title>
<Link rel="stylesheet" th:href="@{/css/common.css}">
<Link rel="stylesheet" th:href="@{/css/header_nav_footer.css}">
<Link rel="stylesheet" th:href="@{/css/home.css}">

</head>
<body>
<header th:replace="~/include/header_nav_footer.html::header"></header>
<nav th:replace="~/include/header_nav_footer.html::nav"></nav>

<section>
<div id="section_wrap">
<div class="word">HOME</div>
<div class="content">

</div>
</div>
</section>

<footer th:replace="~/include/header_nav_footer.html::footer"></footer>

</body>
</html>
```

Flagment를 사용하려면
th:replace 속성을 이용한다.

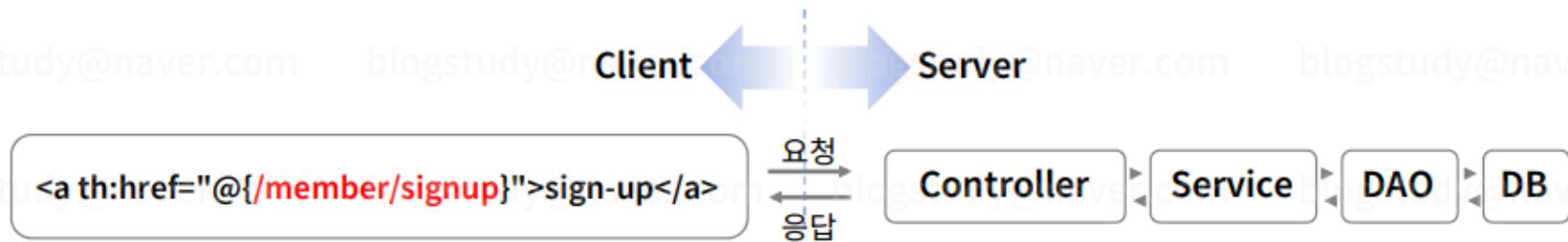
회원가입





spring

➤ 회원가입 양식 컨트롤러 구현





spring

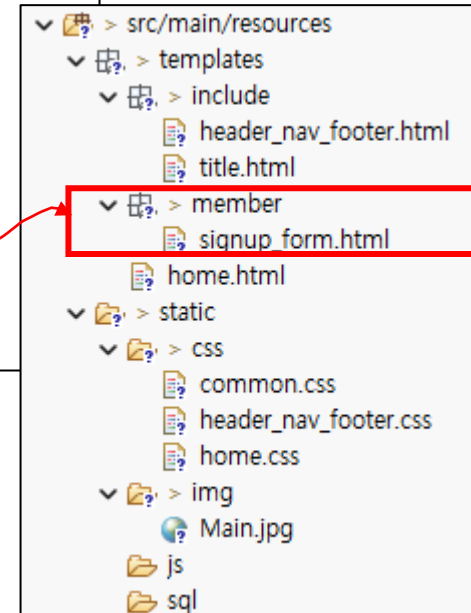
- src -> main -> java -> member 패키지 생성 후 -> MemberController.java 파일생성
 - 단, 반드시 templates -> member 폴더를 생성한 후 signup_form.html파일을 생성한다.
- 이유는 MemberController의 return 의 nextPage의 경로가 같아야하기때문이다.

```
package com.green.member;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class MemberController {

    @GetMapping("/member/signup")
    public String signup() {
        System.out.println("회원가입 핸들러 호출");
        String nextPage = "member/signup_form";
        return nextPage;
    }
}
```





spring

- src -> main -> resources -> templates -> member 폴더를 생성한다.
- templates -> member 폴더를 생성한 후 signup_form.html 파일을 아래와 같이 작성한다.

```
<!DOCTYPE html>
<html lang="ko" xmlns:th="http://www.thymleaf.com">
<head>
<meta charset="UTF-8">
<title th:replace="~/include/title.html::title">Member Join</title>
<Link rel="stylesheet" th:href="@{/css/common.css}">
<Link rel="stylesheet" th:href="@{/css/header_nav_footer.css}">
<Link rel="stylesheet" th:href="@{/css/home.css}">
<Link rel="stylesheet" th:href="@{/css/member.css}">
</head>
<body>
<header th:replace="~/include/header_nav_footer.html::header"></header>
<nav th:replace="~/include/header_nav_footer.html::nav"></nav>

<section>
<div id="section_wrap">
<div class="word">회원가입</div>
<div class="content">
<form th:action="@{/member/signup_confirm}"
name="signup_form"
method="post">
<table width="500" border="1">
<tr height="50">
<td width="150" align="center">아이디</td>
<td width="350" align="center">
<input type="text" name="id" class="content"
placeholder="아이디를 입력하세요">
</td>
</tr>
```

추가

member.css

```
@charset "UTF-8";
section h2{
text-align: center;
}
section table{
border-collapse: collapse;
}
/* section input[type="submit"],
section input[type="reset"]{
width: 100px;
height: 30px;
cursor: pointer;
} */
#section_wrap form table{
margin: 0 auto;
}
#section_wrap form table .content{
width: 300px;
height: 30px;
}
```



spring

➤ src -> main -> resources -> templates -> member 폴더를 생성한다.

```
<tr height="50">
  <td width="150" align="center">비밀번호</td>
  <td width="350" align="center">
    <input type="password" name="pw" class="content"
      placeholder="영문과 숫자만 입력하세요">
  </td>
</tr>
<tr height="50">
  <td width="150" align="center">비밀번호확인</td>
  <td width="350" align="center">
    <input type="password" name="pwchk" class="content"
      placeholder="영문과 숫자만 입력하세요">
  </td>
</tr>
<tr height="50">
  <td width="150" align="center">Email</td>
  <td width="350" align="center">
    <input type="email" name="mail" class="content"
      placeholder="@를 포함하여 입력하세요">
  </td>
</tr>
<tr height="50">
  <td width="150" align="center">전화번호</td>
  <td width="350" align="center">
    <input type="tel" name="phone" class="content"
      placeholder="000-0000-0000의 형식으로 입력하십시오">
  </td>
</tr>
```



spring

- src -> main -> resources -> templates -> member 폴더를 생성한다.

```
<tr height="50">
  <td align="center" colspan="2">
    <input type="button" value="회원가입" onclick="signupForm();">
    <input type="reset" value="취소">
  </td>
</tr>
</table>
</form>
</div>
</div>
</section>

<footer th:replace="~/include/header_nav_footer.html::footer"></footer>
```



spring

- src -> main -> resources -> static -> js 폴더를 생성한다.
- templates -> member 폴더를 생성한 후 signup_form.html의 회원가입할 때 유효성검사를 javascript를 이용해 작성한다.
- js -> member.js 파일을 생성한다.
- Signup_form.html에 외부의 자바스크립트 파일을 thymleaf 방식으로 연결한다.

```
<!DOCTYPE html>
<html lang="ko" xmlns:th="http://www.thymleaf.com">
<head>
  <meta charset="UTF-8">
  <title th:replace="~/include/title.html::title">Member Join</title>
  <Link rel="stylesheet" th:href="@{/css/common.css}">
  <Link rel="stylesheet" th:href="@{/css/header_nav_footer.css}">
  <Link rel="stylesheet" th:href="@{/css/home.css}">
  <Link rel="stylesheet" th:href="@{/css/member.css}">
  <script th:src="@{/js/member.js}"></script>
</head>
```

추가



spring

- src -> main -> resources -> static -> js 폴더를 생성한다.
- js -> member.js 파일에 아래와 같이 유효성 검사코드를 작성한다.

```
/*회원가입 유효성 검사*/
function signupForm() {
    console.log('회원가입폼');

    let form = document.signup_form;
    if (form.id.value === '') {
        alert('새로운 id 입력');
        form.id.focus();
    } else if (form.pw.value === '') {
        alert('새로운 pw 입력!!');
        form.pw.focus();
    } else if (form.mail.value === '') {
        alert('새로운 email 입력!!');
        form.mail.focus();
    } else if (form.phone.value === '') {
        alert('새로운 전화번호 입력!!');
        form.phone.focus();
    } else {
        form.submit();
    }
}
```



spring

- src -> main -> java -> member 패키지 생성 후 -> MemberController.java 파일생성
 - 회원가입 확인 페이지 signup_result 페이지를 연결하는 MemberController에 메소드를 아래와 같이 작성한다.

```
package com.green.member;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class MemberController {

    //회원가입 양식
    @GetMapping("/member/signup")
    public String signup() {
        System.out.println("회원가입 핸들러 호출");
        String nextPage = "member/signup_form";
        return nextPage;
    }

    //회원가입 확인
    @PostMapping("/member/signup_confirm")
    public String signupConfirm() {
        System.out.println("회원가입 확인 핸들러");
        String nextPage = "member/signup_result";
        return nextPage;
    }
}
```



spring

- src -> main -> java -> member 패키지 생성 후 -> MemberDTO.java 파일생성
 - Signup_form.html에서 사용자가 id, pw, email, phone 정보를 입력 하면 signup_result.html로 넘겨 주는데 매개변수로 사용할 수 있는 MemberDTO를 member 폴더 아래 생성한다.
(반드시 getter, setter한다)

```
package com.green.member;

public class MemberDTO {
    private int no; //사용자 고유번호
    private String id; //사용자 아이디
    private String pw; //사용자 비밀번호
    private String mail; //사용자 email
    private String phone; //사용자 전화번호
    private String reg_date; //사용자 정보 등록일
    private String mod_date; //사용자 정보 수정일

    public int getNo() {
        return no;
    }
    public void setNo(int no) {
        this.no = no;
    }
    public String getId() {
        return id;
    }
    public void setId(String id) {
        this.id = id;
    }
}
```



spring

- src -> main -> java -> member 패키지 생성 후 -> MemberController.java 파일생성
 - MemberDTO 생성 후 아래와 같이 signupConfirm() 메소드를 수정한다.

```
package com.green.member;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class MemberController {

    //회원가입 양식
    @GetMapping("/member/signup")
    public String signup() {
        System.out.println("회원가입 핸들러 호출");
        String nextPage = "member/signup_form";
        return nextPage;
    }

    //회원가입 확인
    @PostMapping("/member/signup_confirm")
    public String signupConfirm() {
        System.out.println("회원가입 확인 핸들러");
        String nextPage = "member/signup_result";
        return nextPage;
    }
}
```

```
//회원가입 확인
@PostMapping("/member/signup_confirm")
public String signupConfirm(MemberDTO mdto) {
    System.out.println("회원가입 확인 핸들러");
    // signup_result.html파일로 응답을 처리한다.
    String nextPage = "member/signup_result";
    return nextPage;
}
```



spring

- src -> main -> java -> member 패키지 생성 후 -> MemberService.java 파일생성
 - 반드시 @Service 로 서비스로직임을 명시 하여야 한다.
 - MemberController에 어노테이션 @Autowired를 이용해 MemberService를 DI(의존성 주입) 한다.

```
@Controller
public class MemberController {

    @Autowired
    MemberService memberservice;

    //회원가입 양식
    @GetMapping("/member/signup")
    public String signup() {
        System.out.println("회원가입 핸들러 호출");
        String nextPage = "member/signup_form";
        return nextPage;
    }

    //회원가입 확인
    @PostMapping("/member/signup_confirm")
    public String signupConfirm(MemberDTO mdto) {
        System.out.println("회원가입 확인 핸들러");
        // signup_result.html파일로 응답을 처리한다.
        String nextPage = "member/signup_result";
        //회원가입이 제대로 되었는지, 실패했는지를 확인한다.
        int result = memberservice.signupConfirm(mdto);
        return nextPage;
    }
}
```

```
package com.green.member;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class MemberService {

    @Autowired
    MemberDAO memberDao;

    public int signupConfirm(MemberDTO mdto) {
        System.out.println("MemberService 로그인 확인 메소드 호출 완료");
        return 0;
    }
}
```



spring

- src -> main -> java -> member 패키지 생성 후 -> MemberDAO.java 파일생성
 - 반드시 @Repository로 DAO임을 명시 하여야 한다.
 - MemberService 에 어노테이션 @Autowired를 이용해 MemberDAO를 DI(의존성 주입) 한다.

```
@Service
public class MemberService {

    // 회원가입의 중복을 확인하는 상수
    public final static int user_id_alreday_exist = 0;
    // 회원가입의 성공여부를 확인하는 상수
    public final static int user_signup_success = 1;
    // 회원가입의 실패를 확인하는 상수
    public final static int user_signup_fail = -1;

    @Autowired
    MemberDAO memberDao;

    public int signupConfirm(MemberDTO mdto) {
        System.out.println("MemberService 로그인 확인 메소드 호출 완료");
        // 회원가입 중복체크하는 부분
        boolean isMember = memberDao.isMmeber(mdto.getId());
        // 아이디 중복체크를 통과했다면
        if (isMember == false) {
            int result = memberDao.insertMember(mdto);
            if (result > 0) {
                return user_signup_success;
            } else {
                return user_signup_fail;
            }
        } else {
            return user_id_alreday_exist;
        }
    }
}
```



spring

- src -> main -> java -> member 패키지 생성 후 -> MemberDAO.java 파일생성
 - MemberDAO에 아래와 같이 작성한다.

```
package com.green.member;

import org.springframework.stereotype.Repository;

@Repository
public class MemberDAO {

    public boolean isMmeber(String id) {
        System.out.println("같은 id존재 여부 확인 메소드 호출");
        return false;
    }

    public int insertMember(MemberDTO mdto) {
        System.out.println("회원 추가 메소드 호출");
        return 0;
    }
}
```



spring

- src -> main -> java -> member -> MemberController.java -> signupConfirm 메소드를 아래와 같이 수정한다.
 - signupConfirm()메소드에서 회원가입 여부의 결과인 result 는 Model 객체로 담아서 signup_result에 보내야 한다.

```
//회원가입 확인
@PostMapping("/member/signup_confirm")
public String signupConfirm(MemberDTO mdto, Model model) {
    System.out.println("회원가입 확인 핸들러");
    // signup_result.html파일로 응답을 처리한다.
    String nextPage = "member/signup_result";
    //회원가입이 제대로 되었는지, 실패했는지를 확인한다.
    int result = memberservice.signupConfirm(mdto);
    model.addAttribute("result",result);
    return nextPage;
}
```



spring

- src -> main -> resources -> templates -> member -> signup_result.html 생성

```
<!DOCTYPE html>
<html lang="ko" xmlns:th="http://www.thymleaf.com">
<head>
<meta charset="UTF-8">
<title th:replace="~/include/title.html::title"></title>
<Link rel="stylesheet" th:href="@{/css/common.css}">
<Link rel="stylesheet" th:href="@{/css/header_nav_footer.css}">
<Link rel="stylesheet" th:href="@{/css/member.css}">

</head>
<body>
<header th:replace="~/include/header_nav_footer.html::header"></header>
<nav th:replace="~/include/header_nav_footer.html::nav"></nav>

<section>
  <div id="section_wrap">
    <div class="word">
      회원 가입 결과
    </div>
    <div class="content">

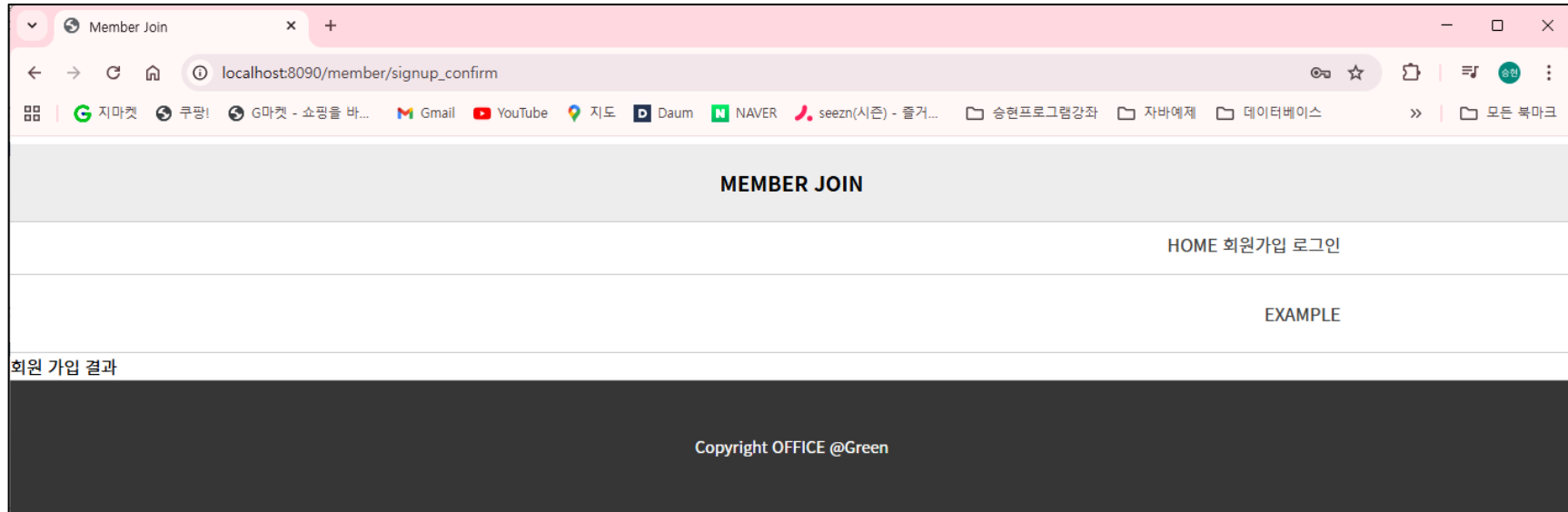
    </div>
  </div>
</section>

<nav th:replace="~/include/header_nav_footer.html::footer"></nav>
</body>
</html>
```



spring

➤ 실행화면





spring

➤ 폴더 구조

```
com.green [boot] [devtools] [pjt main]
├── src/main/java
│   └── com.green
│       ├── member
│       │   ├── MemberController.java
│       │   ├── MemberDAO.java
│       │   ├── MemberDTO.java
│       │   └── MemberService.java
│       ├── Application.java
│       └── HomeController.java
├── src/main/resources
│   ├── templates
│   │   ├── include
│   │   │   ├── header_nav_footer.html
│   │   │   └── title.html
│   │   └── member
│   │       ├── signup_form.html
│   │       ├── signup_result.html
│   │       └── home.html
│   ├── static
│   │   ├── css
│   │   │   ├── common.css
│   │   │   ├── header_nav_footer.css
│   │   │   ├── home.css
│   │   │   └── member.css
│   │   ├── img
│   │   │   └── Main.jpg
│   │   ├── js
│   │   │   └── member.js
│   │   └── sql
│   └── application.properties
```